Topic 5

Labelling

Overview

- Introduction to Edge detection
- Steps in Edge detection
- First derivative operators
  - Gradient
  - Sobel, Robert and Prewitt
- Second derivative operators
  - Laplacian
  - LoG (Laplacian of Gaussian)

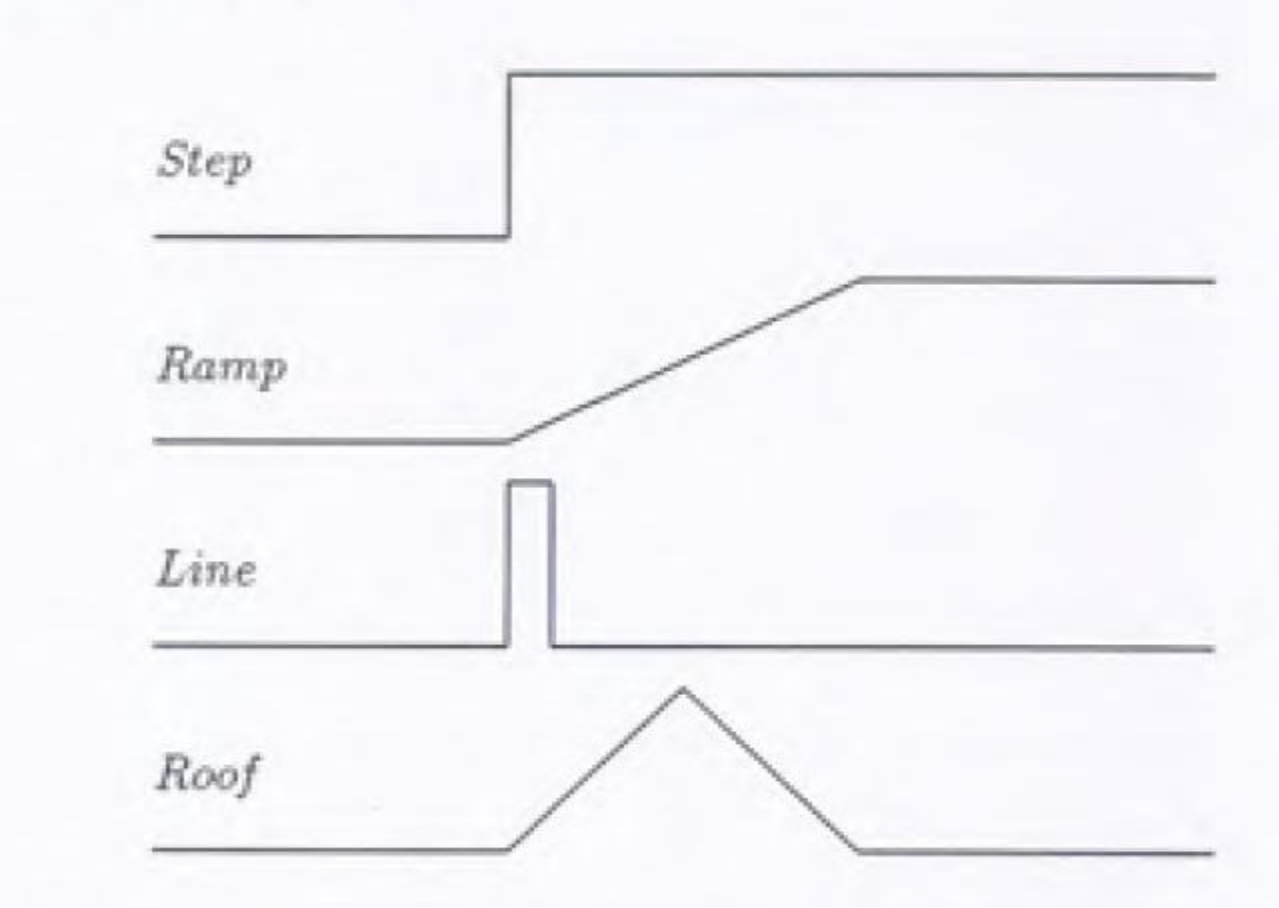
What is Edge Detection?

- In computer vision, edge detection is the process of identifying significant changes in brightness, color, or texture, which indicate the presence of an edge.
- Edge detection is a fundamental image processing technique for identifying and locating the boundaries or edges of objects in an image.
- It involves identifying and locating sharp discontinuities in an image, which typically correspond to significant changes in intensity or color.
- It is used to identify and detect the discontinuities in the image intensity and extract the outlines of objects present in an image.
- These discontinuities are referred to as edges, and they play a crucial role in understanding the structure and contents of an image.

Introduction to Edge Detection

- Edge → are regions with sudden intensity changes — eg, where a dark object meets a light background.
- A set of connected pixels that lie on the boundary between 2 regions
- Significant local change in the image intensity, usually associated with a discontinuity in image intensity
- Discontinuities in image intensity can be either
  - Step edge: intensity abruptly changes
  - Line edge: intensity changes value but return to starting value
- However, step and line edges are rare in real images due to smoothing effect introduced by image acquisition system
- Blurred edges tend to be thick; sharpen edges tend to be thin
- Step edges become ramp edges and line edges become roof edges, where intensity changes are not instantaneous but occur over a finite distance.

Introduction to Edge Detection (cont')



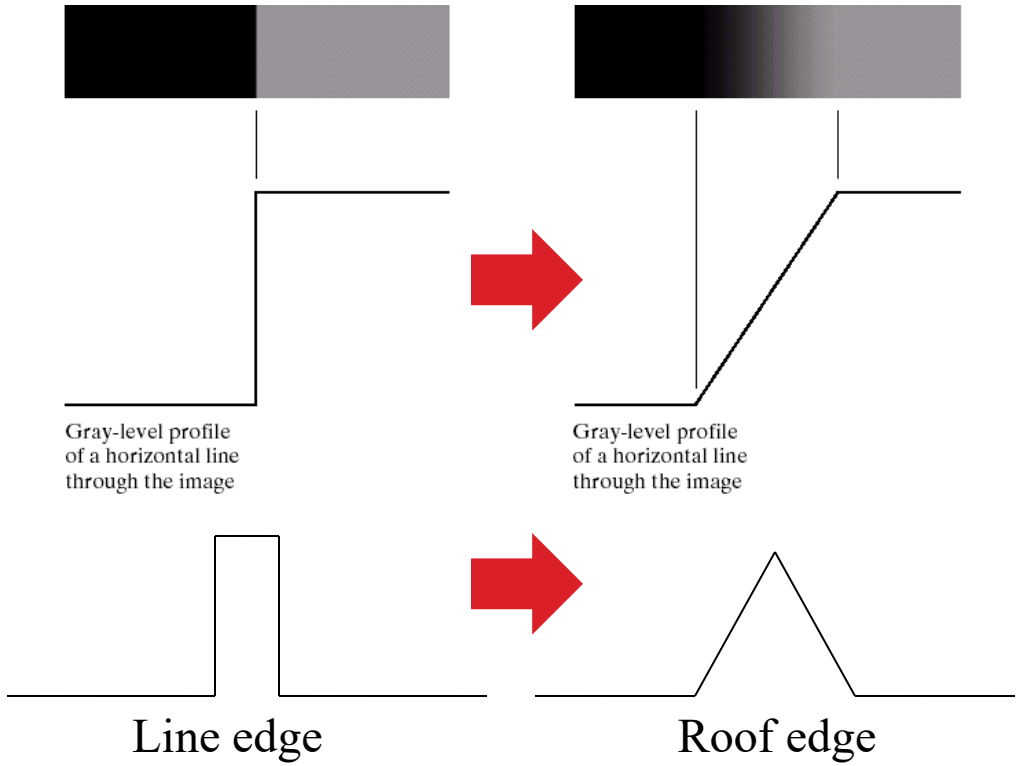
One-dimensional edge profiles

Introduction to Edge Detection (cont')

(a) (b) (c)

From left to right, models (ideal representations) of a step, a ramp, and a roof edges, and their corresponding intensity profiles.

Introduction to Edge Detection (cont')

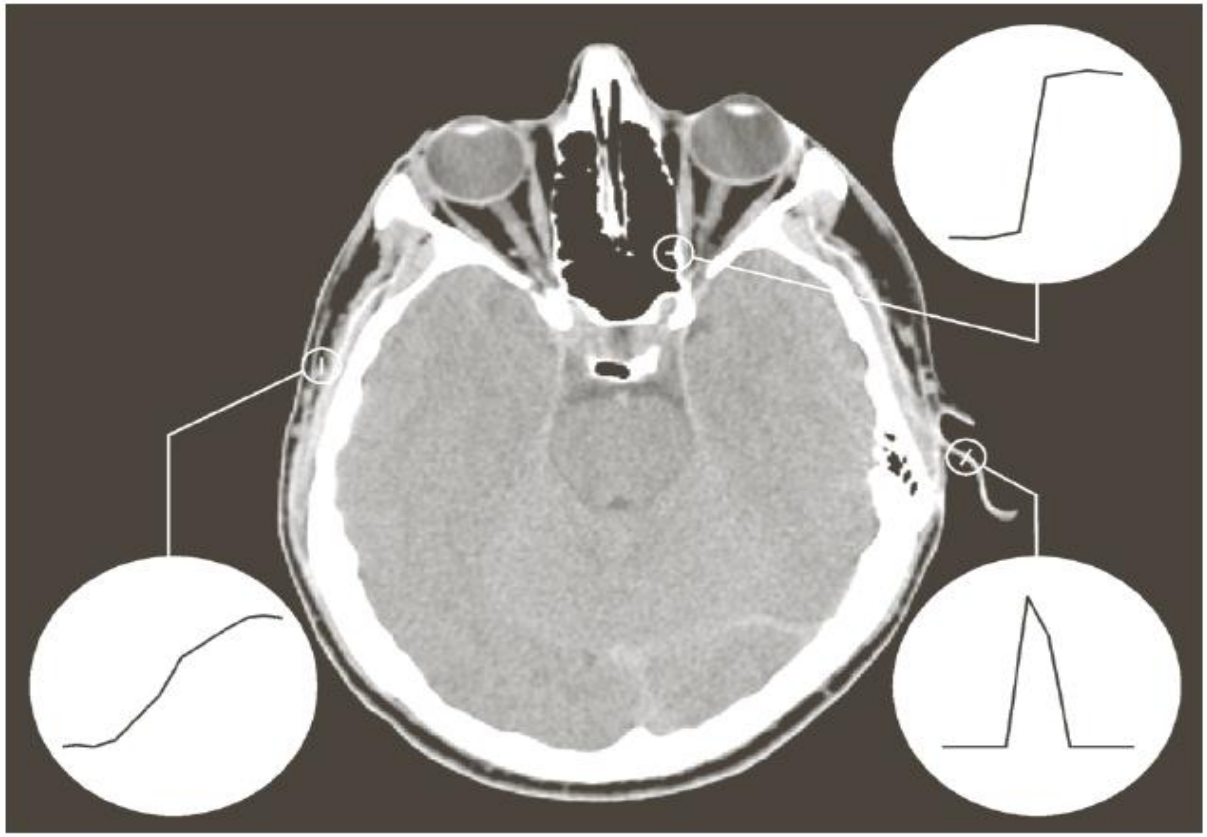


Intensity changes are not instantaneous but occur over a finite distance

Due to:

- image acquisition imperfection
- sampling
- illumination condition

Introduction to Edge Detection (cont')



A 1508 × 1970 image showing (zoomed) actual ramp (bottom, left), step (top, right), and roof edge profiles. The profiles are from dark to light, in the areas indicated by the short line segments shown in the small circles. The ramp and "step" profiles span 9 pixels and 2 pixels, respectively. The base of the roof edge is 3 pixels.

Steps in Edge Detection

- Algorithm for edge detection contain 3 main steps:
  - **Filtering**: edge computation usually are susceptible to noise, filtering is used to improve the performance of edge-detector with respect to noise. Trade off: loss of edge strength
  - **Enhancement**: emphasize pixels where there is a significant changes in local intensity value (neighborhood) and usually performed by computing the gradient or laplacian magnitude
  - **Detection**: determine which points corresponds to edges (point with strong edge contents) for a particular application. Usually, thresholding provides solution for this stage

Derivative Operators

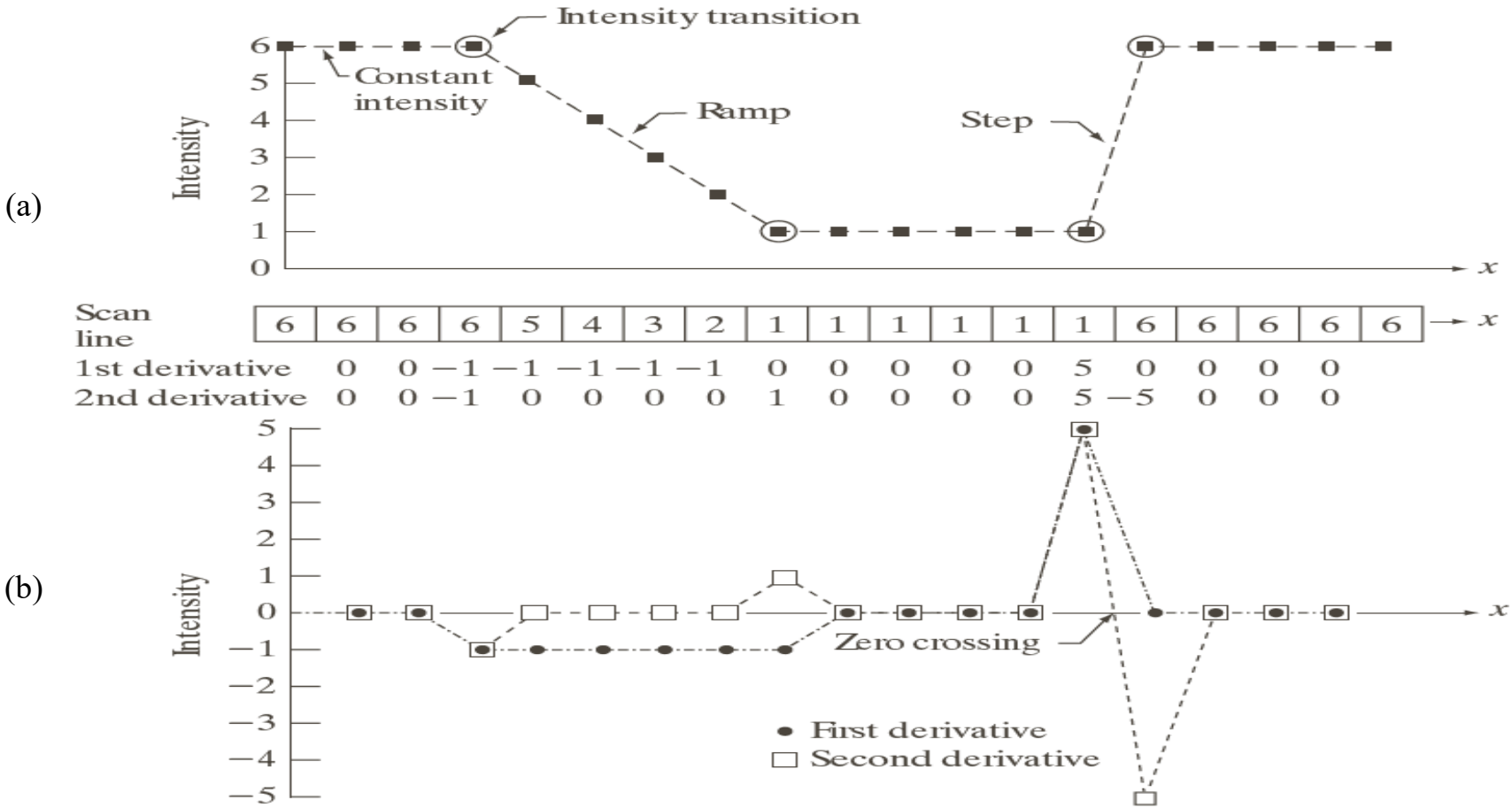
- A basic definition of the first-order derivative of a one-dimensional function $f(x)$ is the difference

$$\boxed{\frac{\partial f}{\partial x} = f(x+1) - f(x)}$$

- The second-order derivative of $f(x)$ as the difference

$$\boxed{\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)}$$

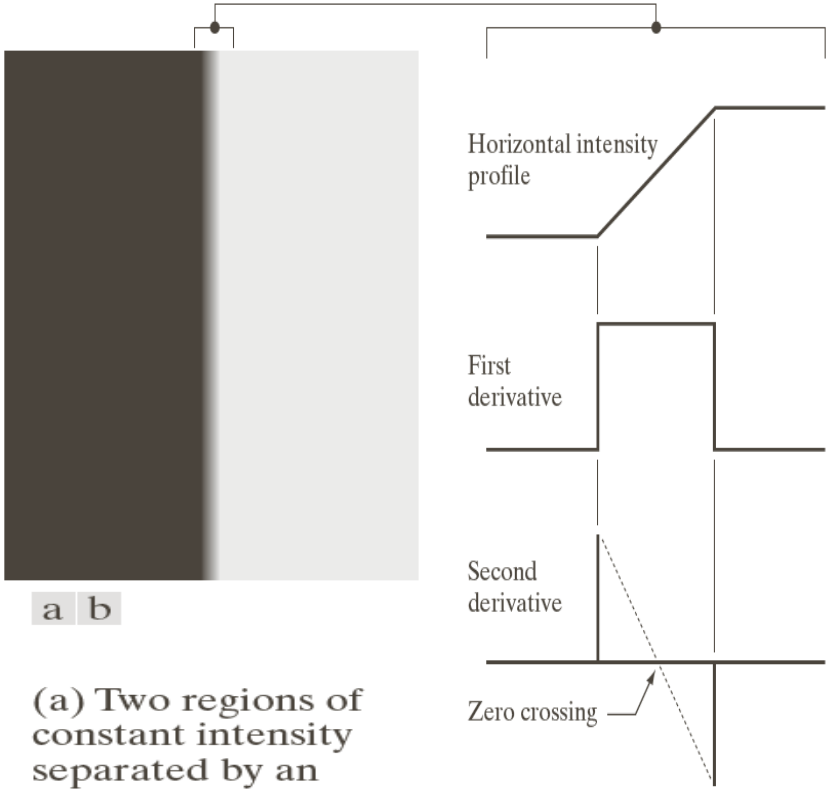
Derivative Operators (cont')



(a) Intensity profile and (b) first and second derivatives of (a).

Derivative Operators (cont')

- First and second derivative properties
  - First order derivative:
    - Magnitude can be used to detect presence of an edge
  - Second order derivative
    - Sign can be used to determine whether edge point lies on the dark or light side of an edge
    - Produce 2 values for every edge of the image (undesirable)
    - Imaginary straight line joining the extreme positive and negative values of second derivative would cross zero near midpoint of the edge (zero-crossing)



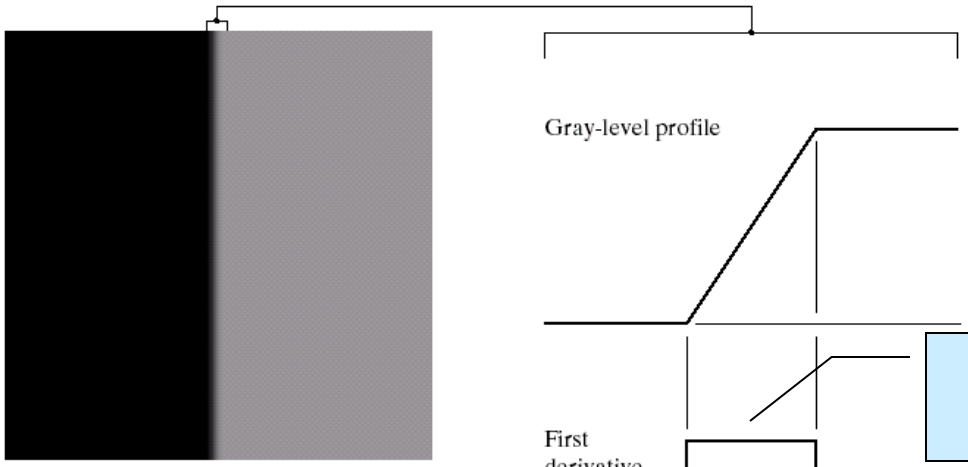
a b

(a) Two regions of constant intensity separated by an ideal vertical ramp edge. (b) Detail near the edge, showing a horizontal intensity profile, together with its first and second derivatives.

Derivative Operators (cont')

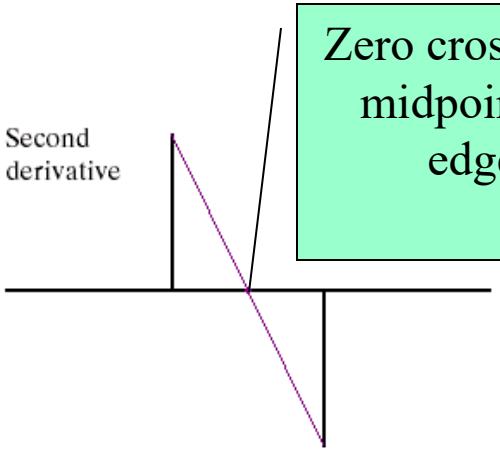
a b

(a) Two regions separated by a vertical edge.
(b) Detail near the edge, showing a gray-level profile, and the first and second derivatives of the profile.



+ve at transition

Zero at constant gray level



Zero crossing – midpoint of edge

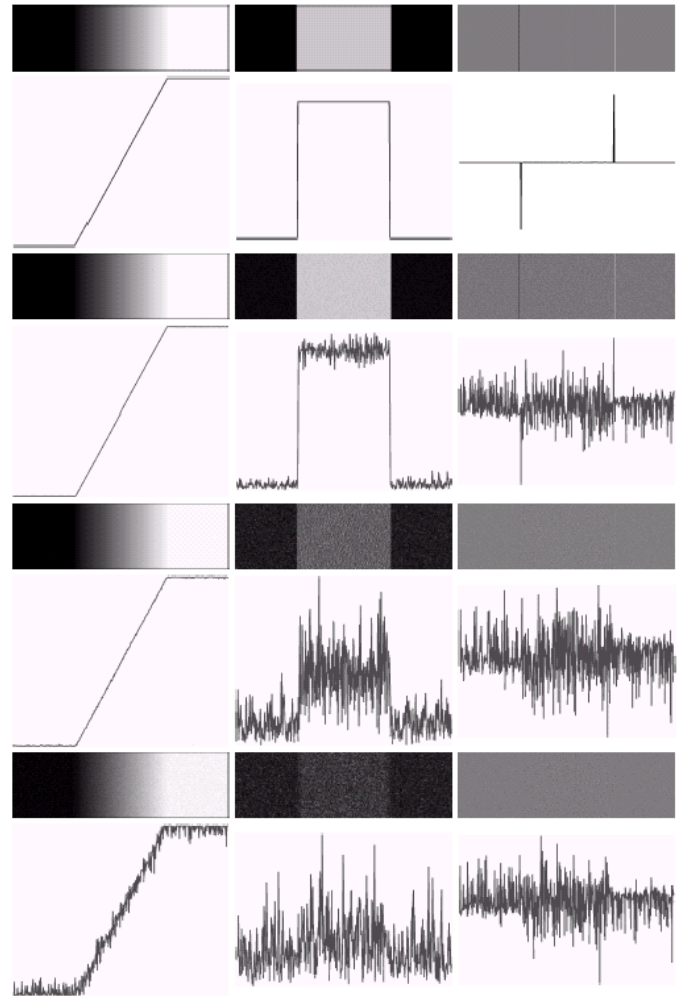
+ve at dark side of edge

-ve at light side of edge

Zero along ramp & constant gray level

Derivative Operators (cont')

- With the presence of noise
  - Little noise can have significant impact on the derivative operators
  - Smoothing is necessary before applying derivative operators



Additive Gaussian noise, mean=0, σ = 0.1, 1.0, 10

First order derivatives

- Gradient operator:
  - The **gradient vector** of an image *f(x, y)* at location *(x,y)* is the vector:

$$\nabla \boldsymbol{f} = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial \boldsymbol{x}} \\ \frac{\partial f}{\partial \boldsymbol{y}} \end{bmatrix} = \begin{bmatrix} f(x+1, y) - f(x, y) \\ f(x, y+1) - f(x, y) \end{bmatrix}$$

  - In edge detection an important quantity is the magnitude of this vector, generally referred to simply as the *gradient*:

$$\boldsymbol{M(x, y) = mag(\nabla f) = \sqrt{G_x^2 + G_y^2}}$$

First order derivatives (cont')

- Common practice is to approximate the gradient with absolute values:

$$\boxed{M(x, y) \approx |G_x| + |G_y|}$$

- The direction of the gradient vector with respect to the *x*-axis can be computed by the equation:

$$\boxed{\alpha(x, y) = \tan^{-1}\left(\frac{G_y}{G_x}\right)}$$

First order derivatives (cont')

- Computation of gradient of an image is based on obtaining the partial derivatives, G_x and G_y at every pixel location
- Consider 3x3 are shown represent the gray levels in a neighborhood of an image. One way to implement gradient at point z_5 is to use *Robert cross-gradient operator:*

$$G_x=(z_9-z_5) \text{ and } G_y=(z_8-z_6)$$

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

- Then the gradient of an image is:

$$M(x, y) \approx |G_x| + |G_y|$$

−1	0
0	1

0	−1
1	0

Roberts

First order derivatives (cont')

- Masks of size 2x2 are awkward to implement due to no clear center
- Approach using masks of size 3x3 are given by *Prewitt* and *Sobel* operators
- *Prewitt* operators:

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

- *Sobel* operators:

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

−1	0	0	−1
0	1	1	0

Roberts

−1	−1	−1	−1	0	1
0	0	0	−1	0	1
1	1	1	−1	0	1

Prewitt

−1	−2	−1	−1	0	1
0	0	0	−2	0	2
1	2	1	−1	0	1

Sobel

First order derivatives (cont')

- *Prewitt* are simpler to implement than *Sobel* masks
- *Sobel* achieves some smoothing (noise suppresion) by giving more importance to the center point (weight of 2 in the center coefficient)

−1	0
0	1

0	−1
1	0

Roberts

−1	−1	−1
0	0	0
1	1	1

−1	0	1
−1	0	1
−1	0	1

Prewitt

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

Sobel

First order derivatives (cont')

- It is possible to modify the 3x3 masks so that they have their strongest responses along the diagonal directions
- *Prewitt* masks for detecting discontinuities in the diagonal direction are shown below

−1	−1	−1
0	0	0
1	1	1

−1	0	1
−1	0	1
−1	0	1

0	1	1
−1	0	1
−1	−1	0

−1	−1	0
−1	0	1
0	1	1

First order derivatives (cont')

- It is possible to modify the 3x3 masks so that they have their strongest responses along the diagonal directions
- *Sobel* masks for detecting discontinuities in the diagonal direction are shown below

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

0	1	2
−1	0	1
−2	−1	0

−2	−1	0
−1	0	1
0	1	2

Sobel

First order derivatives (cont')

- *Example*:

a b
c d

(a) Original image of size 834 × 1114 pixels, with intensity values scaled to the range [0, 1]. (b) $|g_x|$, the component of the gradient in the *x*-direction by using *Sobel* mask. (c) $|g_y|$, obtained using *Sobel* mask in *y*-direction. (d) The gradient image, $|g_x|+|g_y|$

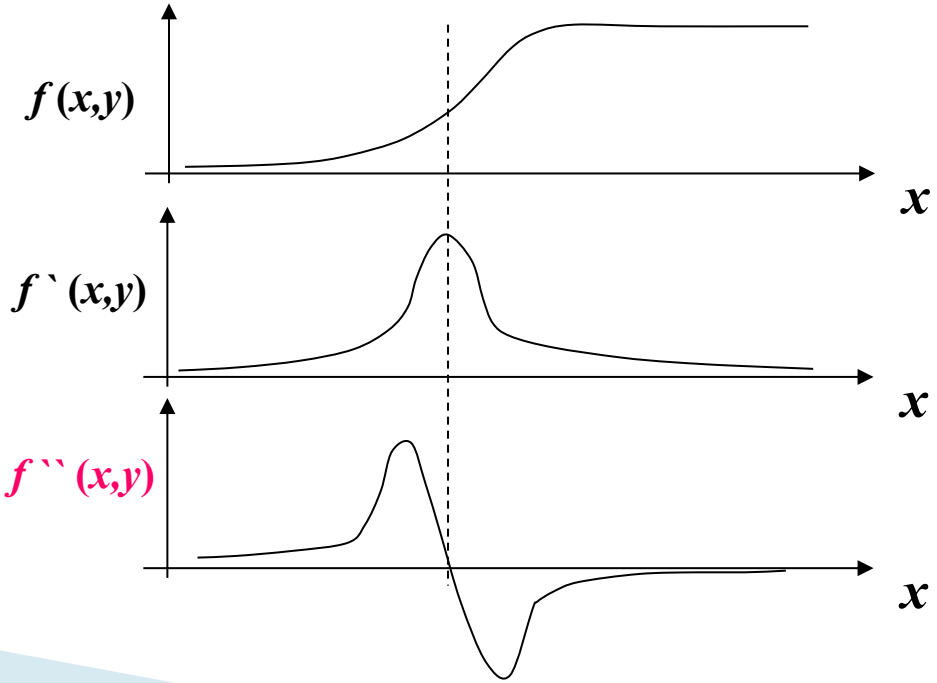
Second order derivatives

- The edge detectors discussed earlier compute the first derivative and if it is above a threshold, the presence of an edge point is assumed
- This will result in detection of too many edge points
- A better approach would be to find only the points that have a local maxima in gradient values and consider them edge points
- This mean that at edge points, there will be a peak in the first derivative and equivalently there will be a zero crossing in the second derivative

Second order derivatives (cont')

- The second derivative of a smoothed step edge is a function that crosses zero at the location of the edge
- The *Laplacian* is the 2D equivalent of the second derivative
- *Laplacian* of a function *f(x,y)* is:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$



Second order derivatives (cont')

- The second derivatives along the x and y directions are approximated using difference equations:

$$\frac{\partial^2 f}{\partial x^2} = \frac{\partial G_x}{\partial x}$$

$$= \frac{\partial(f[x+1,y] - f[x,y])}{\partial x}$$

$$= \frac{\partial f[x+1,y]}{\partial x} + \frac{\partial f[x,y]}{\partial x}$$

$$= (f[x+2,y] - f[x+1,y]) - (f[x+1,y] - f[x,y])$$

$$= f[x+2,y] - 2f[x+1,y] + f[x,y]$$

Second order derivatives (cont')

- However, this approximation is centered about the pixel [$x+1$, y]. Therefore, by replacing x with x-1, we obtain

$$\frac{\partial^2 f}{\partial x^2} = f[x+1, y] - 2f[x, y] + f[x-1, y]$$

- which is the desired approximation to the second partial derivative centered about [i,j]. Similarly for y,

$$\frac{\partial^2 f}{\partial y^2} = f[x, y+1] - 2f[x, y] + f[x, y-1]$$

Second order derivatives (cont')

- By combining these 2 equations into a single operator, the following mask can be used to approximate the *Laplacian*:

$$\nabla^2 \approx \begin{array}{|c|c|c|}\hline 0 & 1 & 0 \\ \hline 1 & -4 & 1 \\ \hline 0 & 1 & 0 \\ \hline \end{array}$$

- Sometimes it is desired to give more weight to the center pixels in the neighborhood. An approximation to the *Laplacian* which does this is

$$\nabla^2 \approx \begin{array}{|c|c|c|}\hline 1 & 4 & 1 \\ \hline 4 & -20 & 4 \\ \hline 1 & 4 & 1 \\ \hline \end{array}$$

Second order derivatives (cont')

- The Laplacian operator signals the presence of an edge when the output of the operator makes a transition through zero.
- Trivial zeros (uniform zero regions) are ignored.
- Consider the example shown in the Figure below:

2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8
2	2	2	2	2	8	8	8	8	8

- The result of the Laplacian filtering on the image with a simple step edge is as shown below (single row):

0	0	0	6	-6	0	0	0

Second order derivatives (cont')

- In the previous example, the zero crossing, corresponding to the edge in the original image, lies halfway between the two center pixels. The edge should be marked at either the left or the right pixel.
- Now, consider another example,

2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8
2	2	2	2	2	5	8	8	8	8

- The result of the Laplacian filtering on the image with a simple step edge is as shown below (single row):

0	0	0	3	0	-3	0	0

Second order derivatives (cont')

- The *Laplacian* operator is not frequently used in machine vision since any operator involving two derivatives is affected by noise more than an operator involving a single derivative.
- Even very small local peaks in the first derivative will result in zero crossings in the second derivative.
- To avoid the effect of noise, powerful filtering methods must be used.
- In the following section, we discuss an approach which combines Gaussian filtering with the second derivative for edge detection.

Laplacian of Gaussian (LoG)

- As mentioned above, edge points detected by finding the zero crossings of the second derivative of the image intensity are very sensitive to noise.
- Therefore, it is desirable to filter out the noise before edge enhancement.
- To do this, the *Laplacian* of Gaussian (LoG), combines Gaussian filtering with the *Laplacian* for edge detection.
- The fundamentals of LoG edge detectors.
  1) The smoothing filter is a Gaussian.
  2) The enhancement is the 2nd derivative (2-D *Laplacian*).
  3) The detection criterion: presence of zero crossing in 2nd derivative with corresponding large peak in 1st derivative.

LoG (cont')

- In this approach, an image is convolved with a Gaussian filter. A Gaussian filter is given by,

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{(-r^2)/(2\sigma^2)}$$

where σ is the standard deviation and $r^2 = x^2 + y^2$.

- Isolated noise points and small structures will be filtered out.
- Since the smoothing will result in spreading of edges, the edge detector considers as edges only those pixels that have locally maximum gradient.
- The Laplacian operator is used as approximation of the second derivative in 2-D.

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

LoG (cont')

- G is the 2-D Gaussian function

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

To find an expression for $\nabla^2 G$ we perform the following differentiations:

$$\nabla^2 G(x,y)=\frac{\partial^2 G(x,y)}{\partial x^2}+\frac{\partial^2 G(x,y)}{\partial y^2}$$

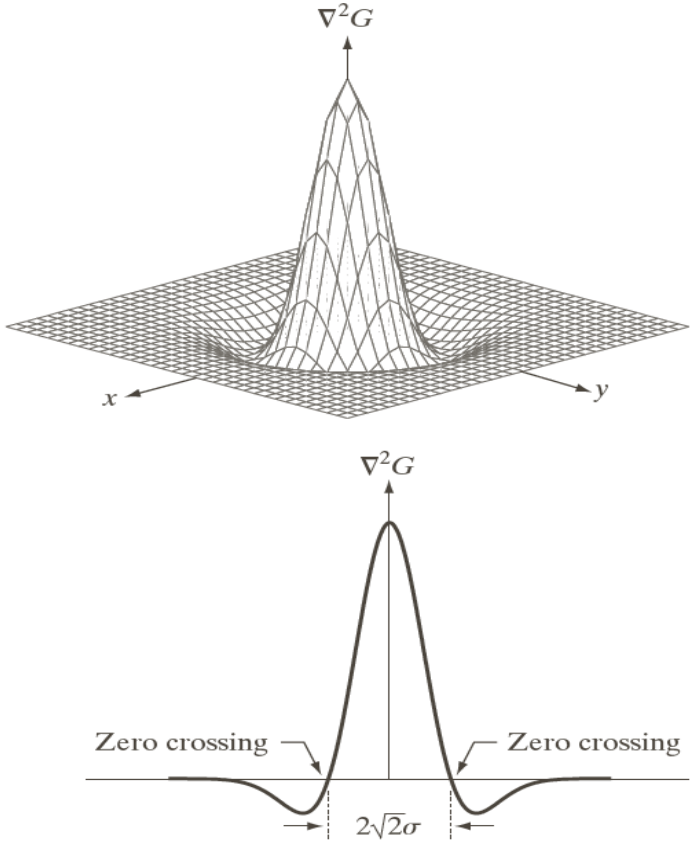
$$=\frac{1}{2\pi\sigma^2}\left(\frac{\partial}{\partial x}\left[\frac{-x}{\sigma^2}e^{\frac{-(x^2+y^2)}{2\sigma^2}}\right]+\frac{\partial}{\partial y}\left[\frac{-y}{\sigma^2}e^{\frac{-(x^2+y^2)}{2\sigma^2}}\right]\right)$$

$$=\frac{1}{2\pi\sigma^2}\left(\left[\frac{x^2}{\sigma^4}-\frac{1}{\sigma^2}\right]e^{\frac{-(x^2+y^2)}{2\sigma^2}}+\left[\frac{y^2}{\sigma^4}-\frac{1}{\sigma^2}\right]e^{\frac{-(x^2+y^2)}{2\sigma^2}}\right)$$

$$=\frac{1}{2\pi\sigma^2}\left(\left[\frac{x^2+y^2-2\sigma^2}{\sigma^4}\right]e^{\frac{-(x^2+y^2)}{2\sigma^2}}\right)\quad=-\frac{1}{\pi\sigma^4}\left(\left[1-\frac{x^2+y^2}{2\sigma^2}\right]e^{\frac{-(x^2+y^2)}{2\sigma^2}}\right)$$

LoG (cont')

- The *Laplacian* of Gaussian Operator, $\nabla^2 G(x, y)$ is commonly known as the *Mexican hat operator* as shown below:



0	0	−1	0	0
0	−1	−2	−1	0
−1	−2	16	−2	−1
0	−1	−2	−1	0
0	0	−1	0	0

a b
c d

(a) Three-dimensional plot of the *negative* of the LoG. (b) Negative of the LoG displayed as an image. (c) Cross section of (a) showing zero crossings. (d) 5 × 5 mask approximation to the shape in (a). The negative of this mask would be used in practice.

LoG (cont')

- A 9x9 discrete mask that approximates this function (for σ = 1.4) is shown in the figure below.

	-4	-3	-2	-1	0	1	2	3	4
-4	0.002	0.008	0.0021	0.0036	0.0043	0.0036	0.0021	0.008	0.002
-3	0.008	0.003	0.007	0.01	0.0108	0.01	0.007	0.003	0.008
-2	0.0021	0.007	0.0112	0.0064	0.0006	0.0064	0.0112	0.007	0.0021
-1	0.0036	0.01	0.0064	-0.0244	-0.0478	-0.0244	0.0064	0.01	0.0036
0	0.0043	0.0108	0.0006	-0.0478	-0.0829	-0.0478	0.0006	0.0108	0.0043
1	0.0036	0.01	0.0064	-0.0244	-0.0478	-0.0244	0.0064	0.01	0.0036
2	0.0021	0.007	0.0112	0.0064	0.0006	0.0064	0.0112	0.007	0.0021
3	0.008	0.003	0.007	0.01	0.0108	0.01	0.007	0.003	0.008
4	0.002	0.008	0.0021	0.0036	0.0043	0.0036	0.0021	0.008	0.002

LoG (cont')

- Generally, the LoG algorithm may be summarized as follows:

  1) Filter the input image with an $n \times n$ Gaussian lowpass filter

  2) Compute the Laplacian of the image resulting from Step 1 using, for example, the 3×3 mask

  3) Find the zero crossings of the image from Step 2

- To implement zero-crossing
  - Threshold the image after applying LoG function
    All positive values → white, all negative → black

  Zero-crossing = transition between black and white

  Drawback:

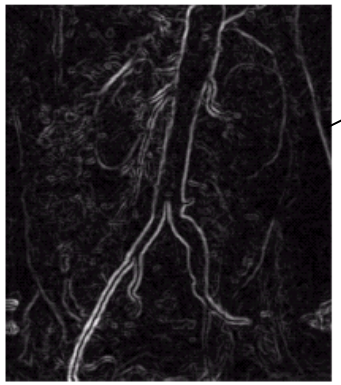
  Spaghetti effect → edges form closed loop

Edge-finding techniques based on various implementation of gradients are used more frequently than zero-crossing

LoG (cont')

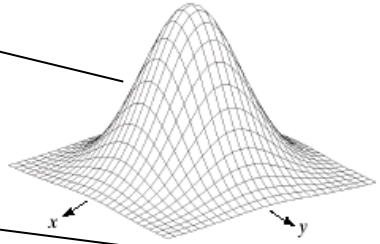
- *LoG + zero-crossings: example*

Original image



Sobel gradient image

Gaussian function



Laplacian mask

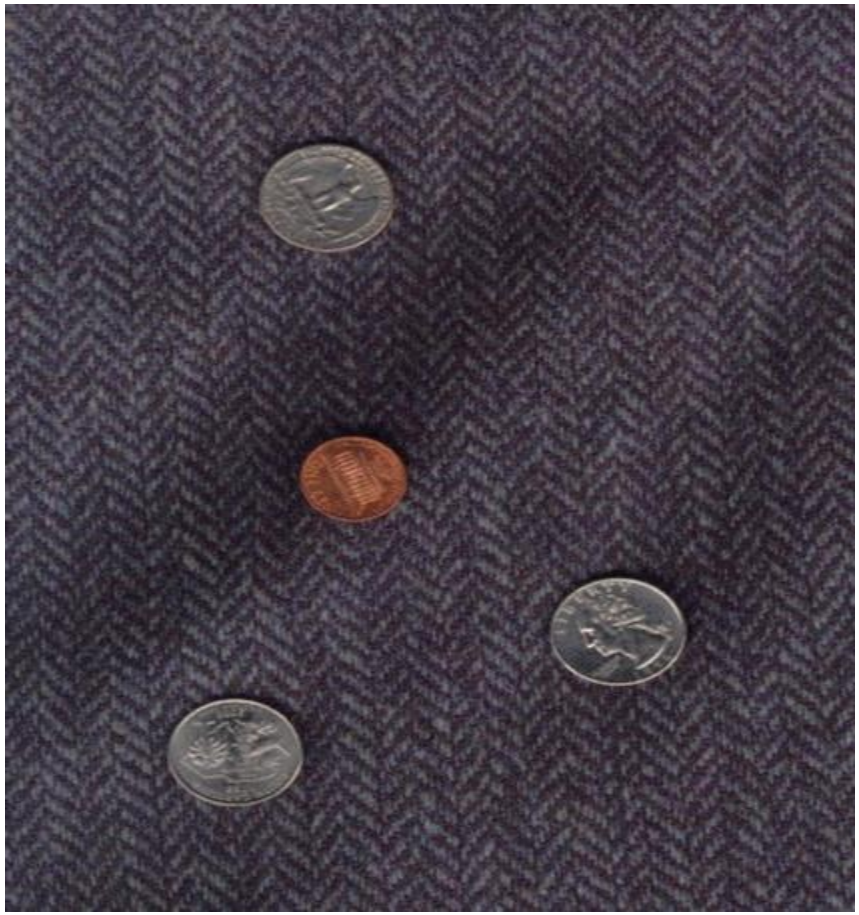
−1	−1	−1
−1	8	−1
−1	−1	−1

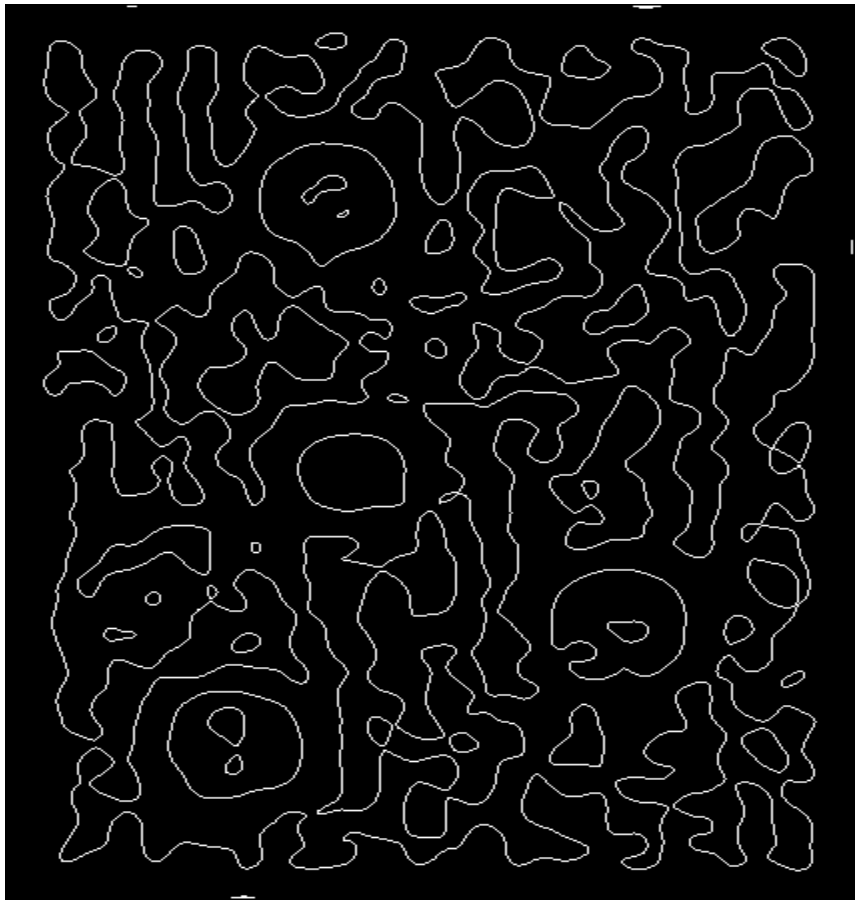
Thresholded image

LoG of image

LoG (cont')

Spaghetti effect - Example





End of Topic 5